Project 13 — Transformer from Scratch

Level: Advanced | **Dataset:** AG News (HuggingFace) | **Framework:** PyTorch

---

Objective

Build a Transformer encoder for text classification entirely from scratch — no pretrained weights.
Cover: positional encoding, multi-head self-attention, masked attention, layer norm, feedforward sublayer, classification head.

Project Structure

```
13_transformer_classification/
├── notebooks/
│   ├── 01_attention_mechanics.ipynb
│   ├── 02_build_transformer.ipynb
│   └── 03_train_evaluate.ipynb
├── data/processed/
│   ├── vocab.pkl
│   └── processed_data.pkl
├── models/model.pkl
├── charts/
├── path_utils.py
├── dashboard_core.py
└── app.py
```

Dataset: AG News (HuggingFace `datasets`)

- 4 classes: World, Sports, Business, Sci/Tech
- 120,000 train, 7,600 test articles
- Short texts (~40 words average) — good for training from scratch

---

Notebook 01 — Attention Mechanics (`01_attention_mechanics.ipynb`)

STOP 1 — Attention: The Core Idea

Write theory cells with examples:

- Attention(Q, K, V) = softmax(QK^T / √d_k) * V
- Q = Query: "what am I looking for?"
- K = Key: "what do I contain?"
- V = Value: "what do I return if matched?"
- **Agent stops here. Explain:**
  - The dictionary analogy: Q is your search query, K is the index, V is the returned content
  - What QK^T computes: similarity score between every query and every key
  - Why divide by √d_k: dot products grow large with high dimension → softmax saturates → tiny gradients
  - What softmax does to the scores: converts to weights that sum to 1
  - What the final weighted sum V produces: a new representation for each position
- Wait for user confirmation before continuing

STOP 2 — Implement Scaled Dot-Product Attention

```python
def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.shape[-1]
    scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))
    weights = F.softmax(scores, dim=-1)
    return torch.matmul(weights, V), weights
```

- Run on a tiny example: 3 tokens, d_k=4
- Print the attention weights matrix [3×3]
- **Agent stops here. Explain:**
  - What the [3×3] attention matrix means: how much each position attends to every other position
  - What masked_fill(-inf) does: after softmax, exp(-inf)=0 → effectively ignores those positions
  - What masking is used for: padding tokens (ignore PAD), causal masking (don't attend to future)
  - How to read an attention matrix: row i, col j = how much token i attends to token j
- Wait for confirmation

STOP 3 — Multi-Head Attention Intuition

- Why multiple heads: different heads learn different types of relationships
- Head 1 might attend to: nearby syntactic relationships
- Head 2 might attend to: long-range coreference
- Head 3 might attend to: semantic similarity
- **Agent stops here. Explain:**
  - How multiple heads work: split d_model into h heads of d_model/h each
  - Each head has its own Wq, Wk, Wv projection matrices
  - All head outputs concatenated, then projected with Wo
  - The intuition: ensemble of attention patterns instead of one
  - Why n_heads must divide d_model evenly (typically d_model=256, n_heads=8 → d_k=32)
- Wait for confirmation

Notebook 02 — Build Transformer (`02_build_transformer.ipynb`)

STOP 4 — Multi-Head Attention Module

python

```python
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        self.d_k = d_model // n_heads
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def forward(self, Q, K, V, mask=None):
        B, L, D = Q.shape
        Q = self.W_q(Q).view(B, L, n_heads, d_k).transpose(1,2)  # [B, H, L, d_k]
        K = self.W_k(K).view(B, L, n_heads, d_k).transpose(1,2)
        V = self.W_v(V).view(B, L, n_heads, d_k).transpose(1,2)
        attn, weights = scaled_dot_product_attention(Q, K, V, mask)
        attn = attn.transpose(1,2).contiguous().view(B, L, D)
        return self.W_o(attn), weights
```

- **Agent stops here. Explain:**
  - The reshape and transpose: split d_model → n_heads × d_k, move heads to batch dim
  - Why `.contiguous()` is needed before `.view()`: transpose doesn't move memory, view needs contiguous storage
  - What W_o does: projects concatenated head outputs back to d_model
  - Parameter count: $4 \times d_model^2$ (W_q, W_k, W_v, W_o)
- Wait for confirmation

STOP 5 — Positional Encoding

```python
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=512, dropout=0.1):
        pe = torch.zeros(max_len, d_model)
        pos = torch.arange(0, max_len).unsqueeze(1)
        div = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
        self.register_buffer('pe', pe.unsqueeze(0))  # [1, max_len, d_model]

    def forward(self, x):
        return self.dropout(x + self.pe[:, :x.size(1)])
```

- **Agent stops here. Explain:**
  - Why Transformer needs positional encoding: self-attention is order-agnostic (permutation invariant)
  - What the sinusoidal formula encodes: unique pattern for each position
  - Why sin for even dims, cos for odd dims: allows the model to learn relative positions via linear combinations
  - Why `register_buffer`: not a parameter (no gradient), but saved in state_dict and moved with .to(device)
  - Alternative: learned positional embeddings (BERT) vs fixed sinusoidal (original Transformer)
- Wait for confirmation

STOP 6 — Feedforward Sublayer

python

```python
class FeedForward(nn.Module):
    def __init__(self, d_model, d_ff, dropout=0.1):
        self.net = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(d_ff, d_model)
        )
```

- Use d_ff = 4 * d_model (standard ratio)
- **Agent stops here. Explain:**
  - Why d_ff = 4 × d_model: expand then contract — expands representational capacity
  - The role of FFN in each Transformer layer: position-wise nonlinear transformation
  - Attention: mixes information across positions; FFN: transforms each position independently
  - Together they form the complete Transformer block
- Wait for confirmation

STOP 7 — Transformer Encoder Layer

python

```python
class EncoderLayer(nn.Module):
    def __init__(self, d_model, n_heads, d_ff, dropout):
        self.attention = MultiHeadAttention(d_model, n_heads)
        self.norm1 = nn.LayerNorm(d_model)
        self.ffn = FeedForward(d_model, d_ff, dropout)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        attn_out, weights = self.attention(x, x, x, mask)  # self-attention: Q=K=V=x
        x = self.norm1(x + self.dropout(attn_out))         # residual + LayerNorm
        ffn_out = self.ffn(x)
        x = self.norm2(x + self.dropout(ffn_out))          # residual + LayerNorm
        return x, weights
```

- **Agent stops here. Explain:**
  - What "self-attention" means: Q=K=V=x, each token attends to all other tokens including itself
  - Why residual connections: allow gradient flow through many layers (same reason as ResNet)
  - Why LayerNorm not BatchNorm for Transformer:
    - LayerNorm normalizes across features per sample (good for variable-length sequences)
    - BatchNorm normalizes across batch (fails for different-length sequences)
  - Pre-norm vs post-norm: this is post-norm (original Transformer); pre-norm is more stable for deep models
- Wait for confirmation

STOP 8 — Full Transformer Classifier

python

```python
class TransformerClassifier(nn.Module):
    def __init__(self, vocab_size, d_model, n_heads, n_layers, d_ff, n_classes, max_
        self.embedding = nn.Embedding(vocab_size, d_model, padding_idx=0)
        self.pos_enc = PositionalEncoding(d_model, max_len, dropout)
        self.layers = nn.ModuleList([EncoderLayer(d_model, n_heads, d_ff, dropout) f
        self.norm = nn.LayerNorm(d_model)
        self.classifier = nn.Linear(d_model, n_classes)

    def forward(self, x, mask=None):
        x = self.pos_enc(self.embedding(x))   # [B, L, d_model]
        for layer in self.layers:
            x, _ = layer(x, mask)
        x = self.norm(x)
        cls_token = x[:, 0, :]                # [CLS] token: first position
        return self.classifier(cls_token)
```

Hyperparameters: `d_model=256, n_heads=8, n_layers=4, d_ff=1024, dropout=0.1`

- **Agent stops here. Explain:**
  - The [CLS] token trick: prepend a special [CLS] token, use its final representation for classification
  - Why [CLS] works: through self-attention, it aggregates information from all other tokens
  - Alternative pooling: mean of all token representations, max pooling
  - Parameter count: embedding + n_layers × (4d_model² + 2d_model×d_ff) + classifier
  - For these hyperparams: roughly 8M parameters
- Wait for confirmation

Notebook 03 — Train & Evaluate (`03_train_evaluate.ipynb`)

STOP 9 — Data Preparation

- `load_dataset('ag_news')`
- Build vocab (25k words, same procedure as Project 07)
- Add `[CLS]` token (index 2), `[PAD]` (0), `[UNK]` (1)
- Encode: prepend [CLS], truncate/pad to MAX_LEN=128
- **Agent stops here. Explain:**
  - Why we need [CLS] in vocab: the Transformer needs it as a special token
  - Why MAX_LEN=128 for AG News: articles are short, 128 tokens captures most content
  - Why 128 instead of 512 (BERT): smaller model, less compute, sufficient for short texts
- Wait for confirmation

STOP 10 — Padding Mask

```python
def create_padding_mask(x, pad_idx=0):
    # 1 where valid, 0 where PAD
    return (x != pad_idx).unsqueeze(1).unsqueeze(2)  # [B, 1, 1, L]
```

- Pass mask to all attention layers
- **Agent stops here. Explain:**
  - What the padding mask prevents: PAD tokens influencing non-PAD token representations
  - The shape [B,1,1,L]: broadcast across query and head dimensions
  - What masked_fill(-inf) does to PAD positions: softmax weight → 0, no attention to PAD
  - Difference from causal mask (used in decoder): padding mask allows all-to-all except PAD positions
- Wait for confirmation

STOP 11 — Training Loop with Warmup

- 10 epochs, AdamW optimizer, CrossEntropyLoss
- LR warmup for first 10% of steps, then cosine decay (same as Project 08)
- `weight_decay=0.01` in AdamW — L2 regularization
- **Agent stops here. Explain:**
  - What AdamW is: Adam + decoupled weight decay
  - Why AdamW not Adam for Transformers: Adam's L2 regularization is incorrect (weight decay applied to adaptive moments), AdamW fixes this
  - What weight_decay=0.01 does: penalizes large weights, prevents overfitting
  - Why warmup is critical for Transformers: attention weights are sensitive to LR at the start
- Wait for confirmation

STOP 12 — Attention Visualization

- After training, extract attention weights from all heads in all layers for one test sample
- Plot: 4×8 = 32 attention heatmaps (4 layers × 8 heads)
- Highlight interesting heads that attend to specific patterns
- **Agent stops here. Explain:**
  - What an attention heatmap shows: [L×L] matrix of which tokens attend to which
  - How to read it: brighter cell = stronger attention
  - What different heads might learn: punctuation heads, positional heads, syntactic heads
  - The "Induction head" pattern: some heads copy previous token patterns (discovered by Anthropic research)
  - Connecting to interpretability: attention visualization is a window into what the model learned
- Wait for confirmation

STOP 13 — Evaluation & Save

- Test accuracy (target: ~90%+ on AG News with this architecture)
- Classification report (4-class F1)
- Confusion matrix
- Save model.state_dict(), vocab, class_names
- Write `predict_topic(text)` → class name + confidence
- **Agent stops here. Explain:**
  - Full inference pipeline: raw text → [CLS] + encode → pad/truncate → padding mask → model → softmax
  - Why AG News is easy enough to get good results with a small from-scratch Transformer
  - How this architecture compares to BERT: BERT has 110M params, 12 layers; ours has ~8M, 4 layers
  - The next step beyond this project: fine-tuning BERT (huggingface Trainer API)
- Wait for confirmation

`dashboard_core.py`

Functions:

- `load_model_and_vocab()` → model, vocab, class_names
- `predict_topic(text)` → class_name, confidence, all_probs
- `get_attention_weights(text)` → attention_matrices [n_layers, n_heads, L, L]
- `get_training_curves()` → dict
- `get_metrics()` → accuracy, per-class F1

`app.py` — Streamlit (~80 lines)

Sections:

1. Text area: user types or pastes a news article
2. "Classify" button → show predicted topic + 4-class confidence bars
3. Tab 1: Training loss + accuracy curves
4. Tab 2: Attention heatmap for the input text (select layer + head)
5. Tab 3: Classification report (4-class F1)

Key Concepts Covered

- Scaled dot-product attention math (QK^T / √d_k)
- Padding mask and causal mask
- Multi-head attention: split d_model → n_heads × d_k, concat, project
- .contiguous() requirement for view after transpose
- Sinusoidal positional encoding
- register_buffer for non-parameter tensors
- Residual connections + LayerNorm (post-norm)
- d_ff = 4 × d_model pattern
- [CLS] token for classification
- AdamW with weight decay
- LR warmup critical for Transformer training
- Attention visualization and interpretation